Lab 5: Working with Subroutines

In this lab, we'll continue looking at flow-control instructions, specifically the ***subroutine*** call instructions. We'll look at a simple subroutine example, and then we'll implement the Fibonacci sequence using a subroutine call.

Working with Subroutines

A ***subroutine*** in assembly is analogous to a ***method*** or ***function*** call in a higher-level language. The purpose is to jump ***away*** from the current code into a subsection or block of code, execute it, and then return to where we first jumped from. This is like calling ***myFunction***(), which executes the code inside ***myFunction*** before ***returning*** to the original position of the ***myFunction***() call.

A couple things to remember before beginning when it comes to subroutines:

- Subroutines will have some ***label*** that identifies them at the start, which is used by the ***call*** instruction to jump to the subroutine code.
- Subroutines will always end in a ***ret*** (for ***return***) instruction that jumps back to the original ***call*** instruction using the PC value stored in the stack.
- You will almost always need a ***second label*** that is used to ***skip past*** the ***subroutine*** block. Subroutines are only normal assembly, and they ***will*** execute sequentially (i.e. "fall through") if allowed. This is avoided by jumping to a label ***after*** the subroutine.

Let's look at a simple example. Below we have our ***C/C++*** code on the left, and our ***assembly*** code (in a ***.S*** file) on the right:

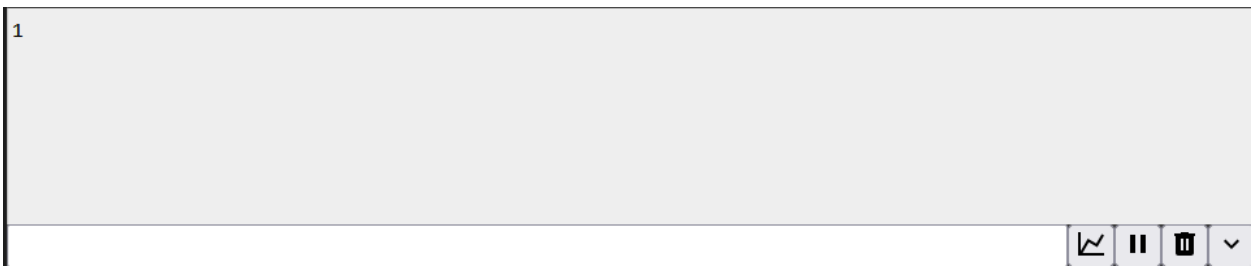
```
extern "C" {
  void myFunction(void);
  extern uint8_t val;
}

void setup() {
  Serial.begin(9600);
  myFunction();
  Serial.println(val);
}

void loop(){
  // Do Nothing
}
```

```
.global myFunction
.global val

.data
val: .BYTE 0

.text
myFunction:
push r22
lds r22, val
call secondFunction
sts val, r22
pop r22
ret

secondFunction:
inc r22
ret
```

In this example code, our main function ***myFunction()*** is called by our ***C/C++*** code. We enter the function, load the value of ***val*** (initialized at 0) into ***r22***, and then we ***call*** our ***secondFunction*** subroutine.

The ***secondFunction*** is defined by its own label further on in the code. It increments the current value of ***r22***, then ***returns***.

Once the subroutine returns, we return to line 12 of the assembly – immediately after we made the ***call*** to the subroutine. We store the current ***r22*** value back into ***val***, pop ***r22*** off the stack, and return the function back to ***C/C++*** land. There we use the serial monitor to print out the ***val*** – it should now be ***one***, since our subroutine incremented it by one when it was executed:



It is also possible to ***nest*** multiple subroutines within each other. For example, this modification of the previous assembly code:

```
16  secondFunction:
17  inc r22
18  call thirdFunction
19  ret
20
21  thirdFunction:
22  inc r22
23  call fourthFunction
24  ret
25
26  fourthFunction:
27  inc r22
28  call fifthFunction
29  ret
30
31  fifthFunction:
32  inc r22
33  ret
34
```

```
1   .global myFunction
2   .global val
3
4   .data
5   val: .BYTE 0
6
7   .text
8   myFunction:
9   push r22
10  lds r22, val
11  call secondFunction
12  sts val, r22
13  pop r22
14  ret
```

Here, the second function calls the third, which calls the fourth, which calls the fifth.

This results in the ***stack*** containing a ***bunch*** of pointers – one for each subroutine call we've made.

Once we reach the last function, it ***returns***, which triggers the return of the next subroutine, and then the next subroutine, and so on – and we clear out all the pointers that were stored in the stack one-by-one (this is also why a ***Last In First Out*** data structure like a stack is used here).

We can also do ***recursion*** by calling our subroutine from within itself; however as with doing recursion in other programming languages, we need to make sure there is some final "end" point or ***base case*** where we ***stop*** recursing and start resolving all the subroutine calls that we generated.

```
.global myFunction
.global val

.data
val: .BYTE 10

.text
myFunction:
push r22
lds r22, val
call secondFunction
sts val, r22
pop r22
ret

secondFunction:
dec r22
tst r22
breq end_recursion
call secondFunction
end_recursion:
ret
```

In this recursive example, we start with ***val = 10***. Every time the ***secondFunction*** subroutine is called, it ***decreases*** val by ***1***, then checks to see if ***val == 0***.

If it ***doesn't*** equal zero, the ***BREQ*** (branch-if-equal) is ***not*** performed, and instead we drop through to make another call to the ***secondFunction*** subroutine.

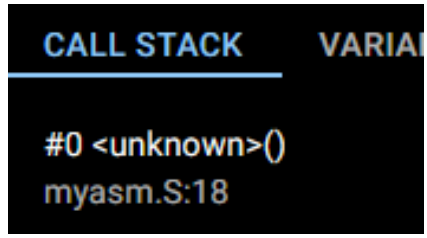
This then starts the process again – decrementing val by one and checking if it's zero. If not, we keep recursively calling the ***secondFunction*** subroutine until it is.

When val finally does equal zero, we take the ***BREQ*** branch to the ***end_recursion*** label, which just causes the ***secondFunction*** subroutine to return. This in turn will "bubble up" through the entire chain of recursion, returning each recursive call that we were waiting on until the stack is clear and we return to ***myFunction*** again (and then finally back into C/C++ land).

Keep in mind that for each ***call*** we make, we're adding another couple of bytes (the pointer to the instruction after the subroutine call) to the ***stack***. If you have ***too much*** recursion, you'll eventually cause the stack to ***overflow.***

For example, if we remove the ***base case*** from the secondFunction subroutine, so that it ***always*** calls the recursive function, and then we try running our code in ***debug mode***, the ***call stack*** will tell us that the current stack counter is ***unknown*** – because we've filled the stack up in an infinite loop!

```
secondFunction:
dec r22
call secondFunction
end_recursion:
ret
```



This will result in your code not printing anything either, as your code will never get past the ***secondFunction*** loop.

Submittable: The Fibonacci Sequence

The ***Fibonacci*** sequence appears in many places in mathematics and nature. Mathematically, it is a function that calculates ***future Fibonacci numbers*** using the ***previous Fibonacci*** numbers:

$$F_0 = 0, \quad F_1 = 1,$$

$$F_n = F_{n-1} + F_{n-2}$$

F0 is 0, and ***F1*** is 1. The third number in the Fibonacci sequence then is ***F0*** + ***F1***, or ***1***. The fourth number in the sequence is ***F2*** + ***F1***, or ***2***. The fifth number in the sequence is ***F3*** + ***F2***, or ***3***. The next number in the sequence is the ***sum*** of the ***previous two numbers*** in the sequence.

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}	F_{16}	F_{17}	F_{18}	F_{19}
0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	610	987	1597	2584	4181

We can use a ***subroutine*** to help us calculate arbitrary Fibonacci numbers (at least until we get to ***F13*** – ***233***, which is the largest Fibonacci number that will fit in an ***8-bit register***).
This subroutine will perform the following steps, in order:

1. Load the previous two Fibonacci values (we will call them F_{n-1} and F_{n-2}) from the ***SRAM*** into two ***registers***.
2. We add those two values together to get the next value F_n in the Fibonacci sequence.
3. We ***store*** the previous F_{n-1} value in the F_{n-2} SRAM memory space (F_{n-2} is no longer required after the addition and can be discarded.)
4. We ***store*** the new F_n value in the F_{n-1} space in the ***SRAM***.
5. We return from the Fibonacci subroutine.

To summarize, we load the two values, add them together, and then store them back into memory, getting rid of the oldest of the three values and only keeping the newer two. That way, if we call the function ***multiple times***, each time it calculates the next number in the sequence and places it in the first ***SRAM*** memory location.

Some simple example assembly to help you get started is provided here. In this case, ***myFunction*** is the routine being called by our ***C/C++*** code, and ***steps*** is a variable to hold the number of Fibonacci iterations (i.e. ***F1***, ***F2***, ***F3*** to calculate).

```
.text
myFunction:
ldi r20, 0
lds r21, steps
loop:
call fibonacci
inc r20
cp r20, r21
brne loop
ret

fibonacci:
// Your Fibonacci Code here
ret
```

For example, if we used step ***6***, your code should print the value ***8*** in the output. If the step was ***13***, it should print ***233***:

```
233
```

Lab Submission

When finished, submit the following files on the course ***Moodle*** for grading:

- ***Lab5_fibonacci.ino*** (C/C++)
- ***Lab5_fibonacci.S*** (Assembly)

I recommend just copy-pasting all the code from the ***Wokwi*** editor into new files in ***Notepad++*** or a similar program. Don't forget to include ***comments*** on your code and include your ***name*** and ***student number*** at the top.